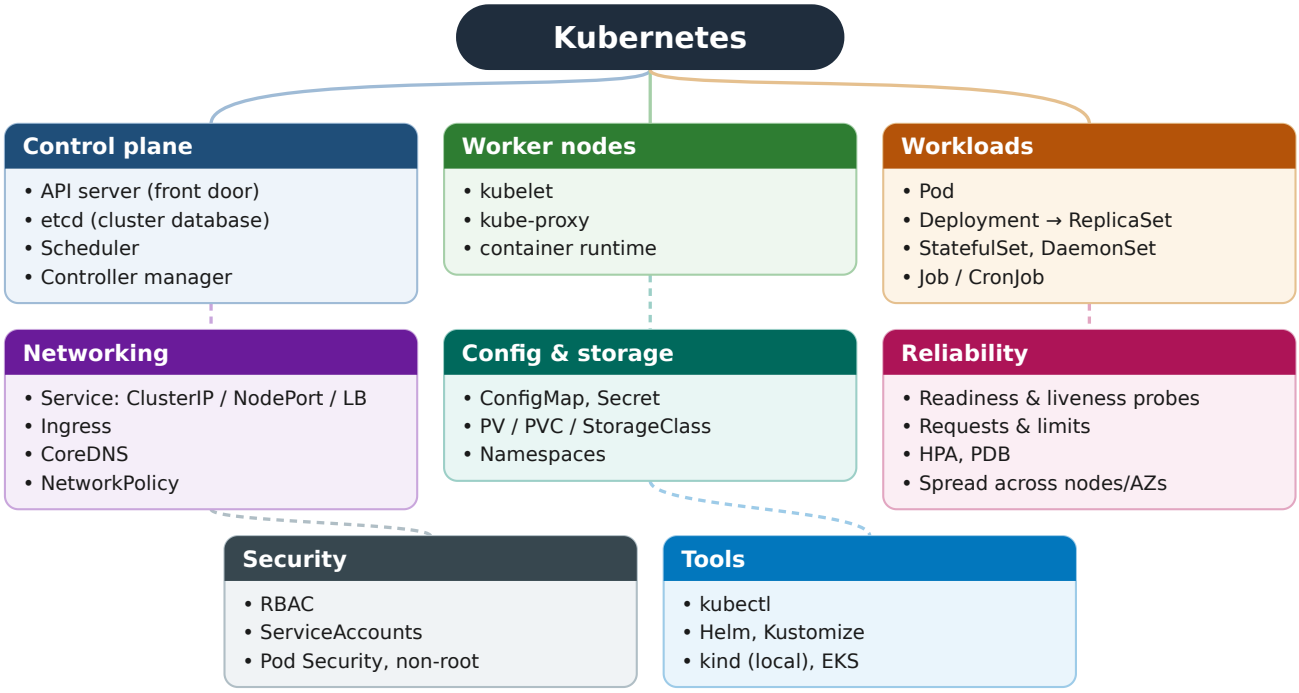
12. Kubernetes

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 12 of 18 · Thirupathi Keloth · v2.0

Contents

12.0 Why Kubernetes
12.1 Core idea: desired state and reconciliation
12.2 Architecture
12.3 Tools and kubectl mastery
12.4 Objects, YAML and metadata
12.5 Pods in depth
12.6 Workload controllers
12.7 Services and networking
12.8 Configuration and secrets
12.9 Storage
12.10 Scheduling and availability
12.11 Autoscaling
12.12 Security
12.13 Packaging: Helm and Kustomize
12.14 Observability on Kubernetes
12.15 Cluster operations
12.16 Amazon EKS in practice
12.17 Complete production application manifests
12.18 Troubleshooting guide
12.19 Hands-on labs (kind on your laptop: free)
12.20 Interview questions
12.21 Self-check



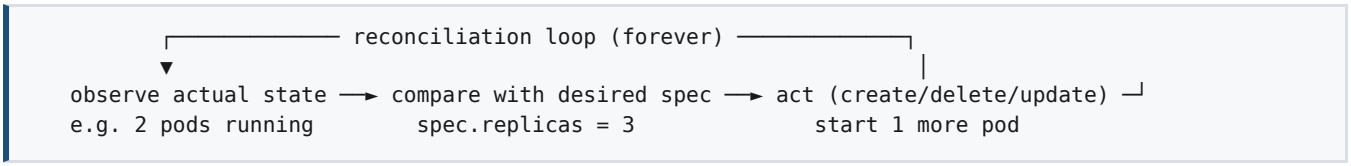
12.0 Why Kubernetes

Docker runs containers on one machine. Real systems run hundreds of containers across many machines and need: placing containers on machines with free capacity, restarting failed ones, scaling up and down, rolling out new versions without downtime, service discovery and load balancing, configuration and secrets, storage, and access control. **Kubernetes (K8s)** is the industry-standard platform that does all of this, on any cloud or on-premises.

Where you meet it: managed clusters (Amazon EKS, Google GKE, Azure AKS), on-prem (OpenShift, Rancher RKE2, vanilla kubeadm), edge (k3s), local development (kind, minikube, Docker Desktop, k3d).

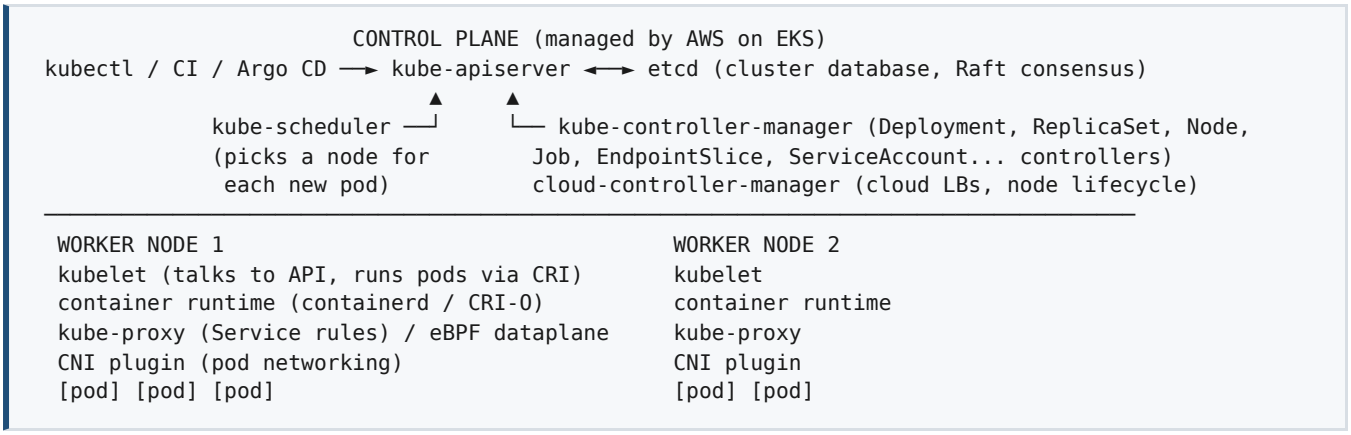
12.1 Core idea: desired state and reconciliation

Kubernetes is **declarative**. You store the **desired state** ("3 replicas of image `shop-api:1.4.2`") in the API; **controllers** constantly compare it with the **actual state** and act to close the gap.



Every object has a `spec` (what you want, written by you) and a `status` (what exists, written by Kubernetes). This is why Kubernetes self-heals: delete a pod and the ReplicaSet controller creates a new one.

12.2 Architecture



Component	Responsibility
kube-apiserver	Front door: REST API, authentication, authorization (RBAC), admission control, validation; the only component that talks to etcd
etcd	Distributed key-value store holding all cluster state. **Back it up** (self-managed clusters)
kube-scheduler	Assigns pending pods to nodes based on resource requests, affinity, taints, spread constraints
kube-controller-manager	Runs the built-in control loops
cloud-controller-manager	Integrates with the cloud: load balancers for Services, node addresses/lifecycle
kubelet	Agent on each node: ensures the pods assigned to it are running, runs probes, reports status
Container runtime	Pulls images and runs containers via the CRI (containerd, CRI-O). Docker Engine itself is no longer used directly by Kubernetes, but Docker-built images work fine
kube-proxy	Programs iptables/IPVS rules so Service IPs reach pods (some CNIs like Cilium replace it with eBPF)
CNI plugin	Gives pods IPs and connectivity (AWS VPC CNI, Calico, Cilium, Flannel)
Add-ons	CoreDNS (DNS), metrics-server (resource metrics), ingress controller, CSI storage drivers

What happens when you `kubectl apply` a Deployment

1. `kubectl` sends the Deployment to the **API server** → authenticated, authorized (RBAC), admission policies applied, stored in **etcd**.
2. **Deployment controller** sees it → creates a **ReplicaSet**.
3. **ReplicaSet controller** creates 3 **Pod** objects (no node yet: `Pending`).
4. **Scheduler** picks a node for each pod (writes `spec.nodeName`).
5. **kubelet** on that node sees the pod → asks the runtime to pull the image and start containers; CNI assigns an IP.
6. kubelet runs **probes**; when the readiness probe passes, the pod becomes **Ready**.
7. **EndpointSlice controller** adds the pod IP to the Service's endpoints → traffic flows.

High-availability control planes run 3+ API server/etcd members across AZs (managed services do this for you).

12.3 Tools and kubectl mastery

Local clusters

Tool	Notes
kind	Kubernetes in Docker containers; multi-node; great for labs/CI (`kind create cluster --name lab`)
minikube	VM/container-based single node with add-ons
k3d / k3s	Lightweight Kubernetes (k3s) in Docker
Docker Desktop	Built-in single-node cluster

```
# kind-config.yaml: 1 control plane + 2 workers, port 80 mapped for an ingress controller
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
    extraPortMappings:
      - containerPort: 80
        hostPort: 8080
  - role: worker
  - role: worker
```

`kind create cluster --name lab --config kind-config.yaml` · `kind delete cluster --name lab`.

kubeconfig

`~/.kube/config` (or `$KUBECONFIG`) holds **clusters** (API URL + CA), **users** (credentials: certs, tokens, exec plugins like `aws eks get-token`), and **contexts** (cluster + user + default namespace).

```
kubectl config get-contexts
kubectl config use-context kind-lab
kubectl config set-context --current --namespace=shop
kubectl config view --minify            # current context only
```

Always check **which cluster** you're on before destructive commands (tools: `kubectx` / `kubens`, a prompt that shows the context).

Essential commands

Command	Purpose
`kubectl get pods [-n ns] [-A] [-o wide]`	List (`-A` all namespaces; wide = node, IP)
`kubectl get deploy,svc,ingress,pvc -n shop`	Several types at once
`kubectl get pod web-7d9 -o yaml`	Full live object
`kubectl describe pod web-7d9`	Details + **Events** (scheduling, image pulls, probe failures)
`kubectl apply -f file.yaml` / `-f dir/` / `-k overlay/`	Create or update declaratively (Kustomize with `-k`)
`kubectl diff -f file.yaml`	Preview changes
`kubectl delete -f file.yaml` / `kubectl delete pod web-7d9`	Delete
`kubectl logs pod [-c container] [-f] [--previous] [--tail=100]`	Container logs; `--previous` = crashed instance
`kubectl logs deploy/web --all-containers`	Logs via a controller
`kubectl exec -it pod -- sh`	Shell inside a container
`kubectl debug -it pod --image=busybox --target=app`	Ephemeral debug container (distroless images)
`kubectl port-forward svc/web 8080:80`	Reach a Service/pod from your machine
`kubectl cp pod:/path ./local`	Copy files
`kubectl top pods / nodes`	CPU/memory usage (needs metrics-server)
`kubectl get events -A --sort-by=.lastTimestamp`	Recent events
`kubectl explain deployment.spec.strategy`	Built-in field documentation
`kubectl api-resources`	All resource types (and short names: po, deploy, svc, ns, cm...)
`kubectl auth can-i create deployments -n shop --as alice`	Check permissions
`kubectl rollout status/history/undo/restart deploy/web`	Manage rollouts
`kubectl scale deploy/web --replicas=5`	Scale manually
`kubectl label pod web-7d9 tier=frontend` / `kubectl annotate ...`	Metadata
`kubectl cordon/drain/uncordon node`	Node maintenance

Output and selection

```
kubectl get pods -l app=web,env!=prod                       # label selectors
kubectl get pods --field-selector status.phase!=Running      # field selectors
kubectl get pods -o jsonpath='{.items[*].metadata.name}'
kubectl get pods -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName,IP:.status.podIP
kubectl get pod web-7d9 -o json | jq '.status.containerStatuses[].restartCount'
kubectl create deployment web --image=nginx:1.27 --replicas=2 --dry-run=client -o yaml > web.yaml
```

Imperative vs declarative: `kubectl create/run/expose/scale` are fine for experiments and generating YAML; production uses **declarative YAML in Git** applied by CI or GitOps (`kubectl apply`, Helm, Argo CD). `kubectl apply --server-side` lets multiple managers own different fields.

Useful extras: **k9s** (terminal UI), **krew** plugin manager (`kubectl krew install ctx ns neat tree`), **stern** (multi-pod log tailing).

12.4 Objects, YAML and metadata

```
apiVersion: apps/v1          # API group/version (core resources use "v1")
kind: Deployment             # object type
metadata:
  name: web                  # unique per kind per namespace
  namespace: shop
  labels:                    # identifying key/values: used by SELECTORS
    app.kubernetes.io/name: web
    app.kubernetes.io/part-of: shop
  annotations:               # non-identifying metadata: tools, descriptions, configs
    owner: team-shop@example.com
spec: {}                     # desired state (structure depends on kind)
# status: written by Kubernetes, never by you
```

Concept	Meaning
Labels	Key/value tags used to **select** objects (Services → pods, Deployments → pods, NetworkPolicies, `kubectl -l`). Recommended keys: `app.kubernetes.io/name`, `/instance`, `/version`, `/component`, `/part-of`, `/managed-by`
Annotations	Arbitrary metadata, not for selection (ingress settings, checksums, build info)
Namespaces	Virtual partitions for teams/apps/environments: scope names, RBAC, quotas, policies. System namespaces: `kube-system`, `kube-public`, `default`. Nodes, PVs, ClusterRoles, StorageClasses are **cluster-scoped**
Owner references	Pods are owned by ReplicaSets, owned by Deployments: deleting the owner garbage-collects children
Finalizers	Keys that block deletion until a controller cleans up external resources (stuck "Terminating" objects often have finalizers)
`resourceVersion`, `generation`	Optimistic concurrency / spec change counter

12.5 Pods in depth

A **Pod** is the smallest deployable unit: one or more containers that share a **network namespace** (same IP, talk over `localhost`), can share **volumes**, and are always scheduled together on one node. Pods are **disposable**: you almost never create them directly; controllers (Deployment, StatefulSet, Job) create and replace them.

Complete pod spec (annotated)

```
apiVersion: v1
kind: Pod
metadata:
  name: api
  labels:
    app.kubernetes.io/name: api
spec:
  serviceAccountName: api                 # identity for RBAC / cloud IAM
  automountServiceAccountToken: false     # only mount a token if the app calls the API
  terminationGracePeriodSeconds: 30       # time between SIGTERM and SIGKILL
  securityContext:                        # pod-level security
    runAsNonRoot: true
    runAsUser: 10001
    fsGroup: 10001                        # group owner of mounted volumes
    seccompProfile:
      type: RuntimeDefault
  initContainers:                         # run to completion, in order, before app containers
    - name: wait-for-db
      image: busybox:1.36
      command: ["sh", "-c", "until nc -z db 5432; do echo waiting; sleep 2; done"]
  containers:
    - name: app
      image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
      imagePullPolicy: IfNotPresent
      ports:
        - name: http
          containerPort: 8080
      env:
        - name: LOG_LEVEL
          value: info
        - name: POD_NAME                  # Downward API: pod facts as env vars
          valueFrom:
            fieldRef:
              fieldPath: metadata.name
      resources:
        requests:                         # used by the scheduler (guaranteed share)
          cpu: 250m
          memory: 256Mi
        limits:                           # hard ceiling
          memory: 256Mi
      startupProbe:                       # protects slow starters
        httpGet:
          path: /healthz
          port: http
        failureThreshold: 30
        periodSeconds: 2
      readinessProbe:                     # in or out of Service endpoints
        httpGet:
          path: /ready
          port: http
        periodSeconds: 5
      livenessProbe:                      # restart if stuck
        httpGet:
          path: /healthz
          port: http
        periodSeconds: 10
        failureThreshold: 3
      lifecycle:
        preStop:                          # give load balancers time to drain
          exec:
            command: ["sleep", "5"]
      securityContext:                    # container-level security
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
```

```
        capabilities:
          drop: ["ALL"]
      volumeMounts:
        - name: tmp
          mountPath: /tmp
  volumes:
    - name: tmp
      emptyDir: {}
```

Pod lifecycle

Phase	Meaning
`Pending`	Accepted, but not running yet: waiting for scheduling, image pull, or volumes
`Running`	Bound to a node, at least one container running
`Succeeded`	All containers exited 0 (Jobs)
`Failed`	All containers terminated, at least one non-zero
`Unknown`	Node not reporting (network or node failure)

Container **states**: `Waiting` (with a reason such as `ContainerCreating`, `ImagePullBackOff`, `CrashLoopBackOff`), `Running`, `Terminated` (with exit code and reason such as `OOMKilled`, `Error`, `Completed`).

Restart policy (`Always` default, `OnFailure`, `Never`): the kubelet restarts failed containers in place with exponential back-off (10s, 20s, 40s … capped at 5 minutes). That back-off is what `CrashLoopBackOff` means.

Conditions (`kubectl describe`): `PodScheduled`, `Initialized`, `ContainersReady`, `Ready`. Only `Ready` pods receive Service traffic.

Probes

Probe	Question	On failure	Typical use
startupProbe	Has the app finished starting?	Restart after `failureThreshold × periodSeconds`	Slow-starting apps (JVM); other probes wait until it succeeds
readinessProbe	Can it take traffic right now?	Removed from Service endpoints (no restart)	Warming caches, dependency briefly down, overload
livenessProbe	Is it alive or deadlocked?	Container **restarted**	Recover from hangs

Probe types: `httpGet` (2xx/3xx = success), `tcpSocket`, `exec` (exit 0), `grpc`. Settings: `initialDelaySeconds`, `periodSeconds`, `timeoutSeconds`, `successThreshold`, `failureThreshold`.

Senior rules: - Liveness must check **only the process itself**, never databases or other services. Otherwise one DB outage restarts every pod and turns a partial outage into a total one. - Readiness may check critical dependencies, but carefully: if all pods go unready at once, the Service has no endpoints. - Use a `startupProbe` instead of a large `initialDelaySeconds`. - No liveness probe is better than a wrong one.

Graceful termination

```
kubectl delete / rollout / drain
  │
  ├─► pod marked Terminating ──► removed from EndpointSlices (in parallel, takes a moment)
  ├─► preStop hook runs (e.g. sleep 5 so load balancers stop sending traffic)
  ├─► SIGTERM sent to PID 1 of each container ──► app stops accepting, finishes in-flight requests
  └─► after terminationGracePeriodSeconds (default 30s) ──► SIGKILL
```

The app must handle **SIGTERM** (and run as PID 1 correctly: use exec-form `CMD` / `ENTRYPOINT` or an init like `tini`). Without that you get dropped requests on every deployment.

Multi-container patterns

Pattern	Example
Init container	Wait for dependencies, run migrations, fetch config, fix permissions
Sidecar	Log shipper, service-mesh proxy (Envoy), secrets agent. Native sidecars: an init container with `restartPolicy: Always` starts before and stops after the app (stable since Kubernetes 1.33)
Ambassador	Local proxy to an external service (e.g. cloud SQL proxy)
Adapter	Converts the app's output (metrics format, logs) for a standard system

Resources and QoS

Term	Meaning
CPU units	`1` = one vCPU/core; `250m` = 0.25 core (millicores)
Memory units	`Mi` / `Gi` (powers of 2: 1Mi = 1,048,576 bytes); `M` / `G` are powers of 10
requests	Reserved amount; the scheduler places pods only where requests fit
limits	Ceiling. CPU over the limit is **throttled**; memory over the limit is **OOMKilled** (exit code 137)

QoS class	Rule	Evicted under node memory pressure
Guaranteed	Every container has requests = limits for CPU and memory	Last
Burstable	At least one request or limit set, not Guaranteed	Middle
BestEffort	No requests or limits at all	First

Production practice: always set CPU and memory **requests**; set a memory **limit** (commonly equal to the request for predictability); many teams **omit CPU limits** for latency-sensitive services because throttling causes latency spikes, while requests still guarantee a fair share. Right-size from real usage (`kubectl top`, Prometheus, VPA recommendations).

Security context essentials

Setting	Effect
`runAsNonRoot: true`, `runAsUser: 10001`	Refuse to run as root
`allowPrivilegeEscalation: false`	Block setuid/sudo-style escalation

Setting	Effect
`readOnlyRootFilesystem: true`	Image filesystem cannot be modified (mount `emptyDir` for `/tmp`)
`capabilities: drop: ["ALL"]`	Remove Linux capabilities (add back only what is needed, e.g. `NET_BIND_SERVICE`)
`seccompProfile: RuntimeDefault`	Restrict system calls
`privileged: true`, `hostNetwork`, `hostPID`, `hostPath`	Effectively node access: forbid for applications

These match the Pod Security Standard **restricted** profile (12.12).

Debugging a pod

```
kubectl describe pod api-7d9f-x2k              # events: scheduling, pulls, probe failures, OOM
kubectl logs api-7d9f-x2k -c app --previous    # logs of the crashed container
kubectl get pod api-7d9f-x2k -o jsonpath='{.status.containerStatuses[0].lastState}'
kubectl exec -it api-7d9f-x2k -- sh            # only if the image has a shell
kubectl debug -it api-7d9f-x2k --image=busybox:1.36 --target=app   # ephemeral container
kubectl debug node/worker-1 -it --image=busybox:1.36               # shell on a node (host at /host)
```

12.6 Workload controllers

Controller	Use for	Key behavior
Deployment	Stateless apps (APIs, web)	Manages ReplicaSets; rolling updates and rollbacks; pods are interchangeable
ReplicaSet	(Used by Deployments)	Keeps N pods matching a selector; don't use directly
StatefulSet	Databases, Kafka, ZooKeeper, Elasticsearch	Stable names (`db-0`, `db-1`), stable per-pod storage, ordered start/stop
DaemonSet	One pod per node: log agents, monitoring agents, CNI, CSI node plugins	New nodes get the pod automatically
Job	Run to completion: migrations, batch processing	Retries (`backoffLimit`), parallelism, completions
CronJob	Scheduled Jobs: backups, reports	Cron schedule, concurrency policy, history limits

Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: shop
spec:
  replicas: 3
  revisionHistoryLimit: 5                # old ReplicaSets kept for rollback
  selector:
    matchLabels:                         # immutable after creation; must match template labels
      app.kubernetes.io/name: api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%                      # extra pods allowed above replicas during rollout
      maxUnavailable: 0                  # never drop below desired capacity
  minReadySeconds: 10                    # pod must stay Ready this long to count as available
  progressDeadlineSeconds: 600           # mark rollout failed after 10 minutes without progress
  template:
    metadata:
      labels:
        app.kubernetes.io/name: api
    spec:
      containers:
        - name: app
          image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /ready
              port: 8080
```

How a rolling update works: changing anything in `spec.template` (image, env, resources) creates a **new ReplicaSet**. The controller scales the new one up and the old one down, step by step, respecting `maxSurge` / `maxUnavailable` and waiting for readiness. Changing only `replicas` does not create a new ReplicaSet.

```
kubectl set image deploy/api app=registry/shop/api:1.4.3 -n shop    # (or change YAML in Git)
kubectl rollout status deploy/api -n shop                           # waits; non-zero on failure
kubectl rollout history deploy/api -n shop
kubectl rollout undo deploy/api -n shop [--to-revision=3]
kubectl rollout restart deploy/api -n shop                          # re-create pods (new config)
kubectl rollout pause / resume deploy/api -n shop
```

`strategy.type: Recreate` stops all old pods before starting new ones (downtime; only for apps that cannot run two versions at once). Canary and blue/green need Argo Rollouts, Flagger, or a service mesh.

Note: pods do **not** restart when a ConfigMap or Secret they use changes. Use a checksum annotation in the template (Helm pattern), `kubectl rollout restart`, Kustomize's hashed generator names, or a reloader controller.

StatefulSet

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: db
  namespace: shop
spec:
  serviceName: db                       # headless Service giving stable DNS names
  replicas: 3
  podManagementPolicy: OrderedReady     # db-0, then db-1, then db-2 (Parallel also possible)
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
          envFrom:
            - secretRef:
                name: db-credentials
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:                 # one PVC per pod: data-db-0, data-db-1, data-db-2
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: gp3
        resources:
          requests:
            storage: 20Gi
```

Guarantees: pod names `db-0..db-2`, DNS `db-0.db.shop.svc.cluster.local`, each pod re-attaches **its own** PVC after rescheduling, ordered scale-down from the highest ordinal. PVCs are **kept** when the StatefulSet is deleted (data safety; configurable with `persistentVolumeClaimRetentionPolicy`).

Senior advice: running databases on Kubernetes is possible (use an **operator**: CloudNativePG, Strimzi for Kafka, ECK for Elasticsearch) but managed services (RDS, Aurora, MSK) are usually the better choice on AWS: backups, patching and failover are handled for you.

DaemonSet

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-agent
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: node-agent
  updateStrategy:
    type: RollingUpdate
  template:
    metadata:
      labels:
        app: node-agent
    spec:
      tolerations:                      # also run on tainted nodes (e.g. control plane)
        - operator: Exists
      containers:
        - name: agent
          image: prom/node-exporter:v1.8.2
          ports:
            - containerPort: 9100
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              memory: 128Mi
```

Job and CronJob

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate-1-4-2
  namespace: shop
spec:
  backoffLimit: 3                       # retries before the Job is Failed
  activeDeadlineSeconds: 600            # hard timeout
  ttlSecondsAfterFinished: 3600         # auto-delete one hour after finishing
  template:
    spec:
      restartPolicy: Never              # Jobs need Never or OnFailure
      containers:
        - name: migrate
          image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
          command: ["python", "-m", "app.migrate"]
---
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-report
  namespace: shop
spec:
  schedule: "30 1 * * *"                # 01:30 every day
  timeZone: Asia/Kolkata                # without it, the controller's time zone (usually UTC)
  concurrencyPolicy: Forbid             # skip a run if the previous one is still running
  startingDeadlineSeconds: 300
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: report
              image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/report:2.0.1
```

Parallel Jobs: `completions: 10` + `parallelism: 3` runs 10 successful pods, 3 at a time; `completionMode: Indexed` gives each pod an index (`JOB_COMPLETION_INDEX`) to process a shard. Jobs must be **idempotent**: a pod can run twice after a node failure.

Choosing a controller

```
Does it run forever?
├── No → once: Job · on a schedule: CronJob
└── Yes → one copy on every node? → DaemonSet
          └── needs stable identity or its own disk per replica? → StatefulSet (or operator)
               └── otherwise → Deployment
```

12.7 Services and networking

The Kubernetes network model

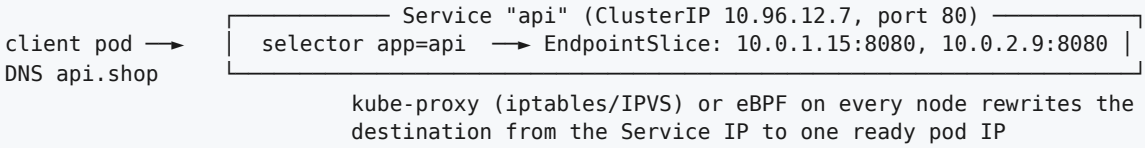
1. Every pod gets its **own IP**.
2. Pods can reach all other pods **without NAT**, across nodes.
3. Agents on a node (kubelet) can reach all pods on that node.
4. Pod IPs are **ephemeral**: a replaced pod gets a new IP. A **Service** gives a stable name and IP.

The **CNI plugin** implements this:

CNI	How it works	Notes
AWS VPC CNI (EKS default)	Pods get real VPC IPs from node ENIs	Native VPC routing and security groups; pod count per node limited by ENIs/IPs (use prefix delegation); plan subnet sizes
Calico	BGP routing or overlay (VXLAN/IP-in-IP)	Strong NetworkPolicy support
Cilium	eBPF dataplane	Can replace kube-proxy; L7 policy; Hubble observability
Flannel	Simple VXLAN overlay	No NetworkPolicy by itself

Three separate IP ranges must never overlap: **node/VPC CIDR**, **pod CIDR** (overlay CNIs), **Service CIDR** (e.g. `10.96.0.0/12`; EKS default `10.100.0.0/16` or `172.20.0.0/16`).

Service types



Type	Reachable from	How
ClusterIP (default)	Inside the cluster	Virtual IP + DNS name
NodePort	`<any node IP>:30000-32767`	ClusterIP plus a port opened on every node
LoadBalancer	Internet or VPC	NodePort/ClusterIP plus a cloud load balancer (on EKS: NLB via AWS Load Balancer Controller)
ExternalName	Inside the cluster	DNS CNAME to an external name (e.g. an RDS endpoint)
Headless (`clusterIP: None`)	Inside the cluster	No virtual IP; DNS returns the pod IPs directly (StatefulSets, client-side load balancing)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: shop
spec:
  type: ClusterIP
  selector:
    app.kubernetes.io/name: api          # sends traffic to READY pods with this label
  ports:
    - name: http
      port: 80                           # Service port (what clients use)
      targetPort: http                   # container port by name or number
      protocol: TCP
```

```
# Internet-facing NLB on EKS (AWS Load Balancer Controller installed)
apiVersion: v1
kind: Service
metadata:
  name: api-public
  namespace: shop
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing
    service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip
spec:
  type: LoadBalancer
  loadBalancerClass: service.k8s.aws/nlb
  selector:
    app.kubernetes.io/name: api
  ports:
    - port: 443
      targetPort: 8080
```

Other fields: `sessionAffinity: ClientIP` (sticky by client IP), `externalTrafficPolicy: Local` (keep the client source IP, only nodes with local pods receive traffic), `internalTrafficPolicy`, `trafficDistribution: PreferClose` (prefer same-zone endpoints, saving cross-AZ cost).

EndpointSlices list the ready pod IPs and ports behind a Service. `kubectl get endpointslices -l kubernetes.io/service-name=api -n shop`. **Empty endpoints = the selector matches no ready pod**: the most common Service problem.

kube-proxy modes: `iptables` (default; rules per Service/endpoint), `ipvs` (kernel load balancer, better for very many Services), `nftables` (newer Linux backend). eBPF CNIs (Cilium) can replace kube-proxy entirely.

DNS (CoreDNS)

Name	Resolves to
`api`	Service `api` in the **same** namespace
`api.shop`	Service `api` in namespace `shop`
`api.shop.svc.cluster.local`	Fully qualified Service name
`db-0.db.shop.svc.cluster.local`	One StatefulSet pod through its headless Service

Pods get `/etc/resolv.conf` with `nameserver <CoreDNS ClusterIP>`, search domains (`shop.svc.cluster.local svc.cluster.local cluster.local`) and `ndots:5`. With `ndots:5`, a name like `api.example.com` (fewer than 5 dots) is first tried with every search suffix: extra lookups. For external names in hot paths, use a trailing dot (`api.example.com.`) or lower `ndots` via `dnsConfig`. **NodeLocal DNSCache** reduces latency and CoreDNS load in large clusters.

```
kubectl run -it --rm dnstest --image=busybox:1.36 --restart=Never -- nslookup api.shop
kubectl -n kube-system logs deploy/coredns
```

Ingress

An **Ingress** is an HTTP(S) routing rule (host and path → Service). It does nothing without an **ingress controller**: ingress-nginx, Traefik, HAProxy, **AWS Load Balancer Controller** (creates an ALB), Istio, Kong.

```
Internet ──► Load balancer ──► Ingress controller ─┬─► shop.example.com/api ──► Service api ──► pods
                                                   └─► shop.example.com/    ──► Service web ──► pods
```

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  namespace: shop
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS":443}]'
    alb.ingress.kubernetes.io/certificate-arn: >-
      arn:aws:acm:ap-south-1:123456789012:certificate/1111-2222
spec:
  ingressClassName: alb                  # which controller handles it
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80
```

With ingress-nginx, TLS comes from a Secret (`spec.tls` with `secretName`) usually issued by **cert-manager** (Let's Encrypt); with the ALB controller, from ACM. **ExternalDNS** creates Route 53 records from Ingress/Service hostnames.

Note: the community **ingress-nginx** project announced its retirement (maintenance ends March 2026). New platforms choose the **Gateway API** with a maintained implementation, or a supported controller.

Gateway API (successor to Ingress)

Role-oriented and more expressive (header matching, traffic splitting, TCP/UDP/gRPC routes, cross-namespace delegation):

Resource	Owned by	Purpose
`GatewayClass`	Infrastructure provider	Which implementation (Envoy Gateway, Cilium, Istio, AWS Gateway API Controller, NGINX Gateway Fabric)
`Gateway`	Platform team	Listeners: ports, protocols, TLS, which namespaces may attach routes
`HTTPRoute` / `GRPCRoute` / `TLSRoute`	App teams	Routing rules to Services

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: api
  namespace: shop
spec:
  parentRefs:
    - name: public-gateway
      namespace: gateway-system
  hostnames: ["shop.example.com"]
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /api
      backendRefs:                        # 90/10 canary split
        - name: api-stable
          port: 80
          weight: 90
        - name: api-canary
          port: 80
          weight: 10
```

NetworkPolicy

By default **every pod can talk to every pod** in the cluster. NetworkPolicies are pod-level firewalls, enforced by the CNI (Calico, Cilium, AWS VPC CNI with network policy enabled; Flannel alone does **not** enforce them).

Rules: - Policies are **additive allow lists**. Once any policy selects a pod for a direction (Ingress/Egress), everything not allowed in that direction is denied. - `podSelector: {}` selects all pods in the namespace. - Selectors in `from` / `to`: `podSelector`, `namespaceSelector`, both in one item (AND), or `ipBlock`.

```
# 1. Default deny all traffic in the namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: shop
spec:
  podSelector: {}
  policyTypes: ["Ingress", "Egress"]
---
# 2. Allow DNS for every pod (otherwise nothing can resolve names)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns
  namespace: shop
spec:
  podSelector: {}
  policyTypes: ["Egress"]
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
---
# 3. Only the ingress controller may reach the API pods on 8080
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-from-ingress
  namespace: shop
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: api
  policyTypes: ["Ingress"]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ingress
      ports:
        - protocol: TCP
          port: 8080
---
# 4. API pods may reach the database (port 5432) only
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-to-db
  namespace: shop
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: api
  policyTypes: ["Egress"]
  egress:
    - to:
```

```
        - ipBlock:
            cidr: 10.0.20.0/24          # database subnet (RDS)
      ports:
        - protocol: TCP
          port: 5432
```

Common mistake: a namespace-and-pod selector written as **two list items** (`- namespaceSelector` and `- podSelector`) means **OR** (much wider) instead of AND.

Service mesh (when you need it)

A mesh (Istio, Linkerd, Cilium service mesh) adds, without app changes: **mutual TLS** between services, retries/timeouts/circuit breaking, fine-grained traffic splitting, and per-request metrics/tracing. Cost: more components, more latency and operational complexity. Adopt it when you have many services and a concrete need (zero-trust mTLS, advanced traffic control), not by default. Istio **ambient mode** removes per-pod sidecars.

12.8 Configuration and secrets

ConfigMap

Non-secret configuration, kept out of the image so the **same image** runs in every environment.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: api-config
  namespace: shop
data:
  LOG_LEVEL: info
  FEATURE_CHECKOUT_V2: "true"           # values are always strings: quote booleans/numbers
  app.yaml: |
    cache:
      ttl_seconds: 300
```

```
# Using it in a container (fragment of a pod spec)
containers:
  - name: app
    image: shop/api:1.4.2
    envFrom:
      - configMapRef:
          name: api-config              # every key becomes an env var
    env:
      - name: LOG_LEVEL                 # or pick single keys
        valueFrom:
          configMapKeyRef:
            name: api-config
            key: LOG_LEVEL
    volumeMounts:
      - name: config
        mountPath: /etc/api             # each key becomes a file: /etc/api/app.yaml
        readOnly: true
volumes:
  - name: config
    configMap:
      name: api-config
      items:
        - key: app.yaml
          path: app.yaml
```

Consumed as	Updates when the ConfigMap changes?
Environment variables	**No**: only at container start (restart needed)
Mounted volume	Yes, eventually (kubelet sync, up to about a minute); the app must re-read the file. Not with `subPath` mounts

`immutable: true` on a ConfigMap/Secret prevents accidental changes and reduces API server load. Limit: 1 MiB per object.

Secret

```
kubectl create secret generic db-credentials -n shop \
  --from-literal=DB_USER=shop --from-literal=DB_PASSWORD="$(openssl rand -base64 24)"
kubectl create secret tls shop-tls --cert=tls.crt --key=tls.key -n shop
kubectl create secret docker-registry regcred --docker-server=REGISTRY \
  --docker-username=USER --docker-password="$TOKEN" -n shop
```

Fact	Consequence
Values are **base64-encoded, not encrypted**	Anyone who can read the Secret (RBAC `get` / `list` secrets) sees the value. `base64 -d` is not security
Stored in etcd	Enable **encryption at rest** (EKS: KMS envelope encryption is on by default for new clusters since 2024)
Never commit Secret YAML to Git	Use one of the patterns below
Mounted secrets are tmpfs (in memory)	Not written to node disk
Types	`Opaque`, `kubernetes.io/tls`, `kubernetes.io/dockerconfigjson`, `kubernetes.io/service-account-token`

Prefer mounting secrets as **files** over environment variables: env vars leak more easily (crash dumps, `env` in debug sessions, child processes).

Production secret management patterns

Pattern	How it works	Good for
External Secrets Operator (ESO)	`ExternalSecret` CR references a key in AWS Secrets Manager / SSM Parameter Store / Vault; ESO creates and refreshes the Kubernetes Secret	Most cloud teams; GitOps-friendly (only the reference is in Git)
Secrets Store CSI Driver	Mounts secrets from the provider directly as files (optionally syncs to a Secret)	Avoid storing in etcd
Sealed Secrets	Encrypt with the cluster's public key (`kubeseal`); only the in-cluster controller can decrypt; the encrypted YAML is safe in Git	Small teams, no external store
SOPS (with KMS/age)	Encrypt values in files; decrypted by Argo CD/Flux plugins or CI	GitOps with existing KMS
Vault Agent / Vault Secrets Operator	Dynamic, short-lived credentials (e.g. per-pod DB users)	Strict security requirements

```
# External Secrets Operator: pull "prod/shop/db" from AWS Secrets Manager
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: db-credentials
  namespace: shop
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager           # a ClusterSecretStore using IRSA/Pod Identity
    kind: ClusterSecretStore
  target:
    name: db-credentials                # Kubernetes Secret ESO creates
  data:
    - secretKey: DB_PASSWORD
      remoteRef:
        key: prod/shop/db
        property: password
```

12.9 Storage

Container filesystems are **ephemeral**: when a container restarts, written files are gone.

Volume types

Volume	Lifetime	Use
`emptyDir`	Pod lifetime (`medium: Memory` for tmpfs)	Scratch space, sharing files between containers, `/tmp` with read-only root
`configMap` / `secret` / `projected` / `downwardAPI`	Pod	Config files, tokens, pod metadata
`hostPath`	Node	Node agents only; a security risk for apps (forbidden by Pod Security baseline)
persistentVolumeClaim	Independent of pods	Databases, uploads, anything that must survive
Ephemeral CSI / generic ephemeral volumes	Pod	Per-pod scratch disks from a StorageClass

PV, PVC and StorageClass

```
Developer: PersistentVolumeClaim "I need 20Gi, ReadWriteOnce, class gp3"
   │
   ▼  dynamic provisioning
StorageClass gp3 (provisioner ebs.csi.aws.com, parameters type=gp3, encrypted=true)
   │
   ▼  CSI driver calls the cloud API
PersistentVolume (an actual 20Gi EBS volume) ── bound 1:1 ── PVC ── mounted by a pod
```

Object	Who creates it	What it is
StorageClass	Platform team	A "type" of storage: provisioner + parameters + policies
PersistentVolumeClaim (PVC)	App team (namespaced)	A request for storage

Object	Who creates it	What it is
PersistentVolume (PV)	Usually auto-created by the CSI driver (cluster-scoped)	The real disk/share

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  encrypted: "true"
reclaimPolicy: Delete                    # Retain for important data
volumeBindingMode: WaitForFirstConsumer  # create the disk in the AZ where the pod is scheduled
allowVolumeExpansion: true
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: uploads
  namespace: shop
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: gp3
  resources:
    requests:
      storage: 20Gi
```

Access modes and reclaim policies

Access mode	Meaning	Examples
ReadWriteOnce (RWO)	Read-write by pods on **one node**	AWS EBS, Azure Disk, GCE PD
ReadWriteOncePod	Read-write by **one pod** only	CSI volumes that need strict single writer
ReadOnlyMany (ROX)	Read-only by many nodes	Shared static content
ReadWriteMany (RWX)	Read-write by many nodes	AWS **EFS**, NFS, CephFS, FSx

Reclaim policy	When the PVC is deleted
`Delete`	The PV and the cloud disk are deleted (default for dynamic provisioning)
`Retain`	PV and disk are kept for manual recovery; use for production data

AWS storage on EKS

Driver	Backing	Notes
EBS CSI driver (EKS add-on)	EBS volumes (gp3, io2)	RWO; a volume lives in **one AZ**, so its pod can only run in that AZ (use `WaitForFirstConsumer`; spread StatefulSet replicas across AZs)
EFS CSI driver	EFS file systems	RWX across AZs; good for shared files, slower per operation than EBS
FSx for Lustre / NetApp ONTAP CSI	FSx	High-performance or enterprise NAS needs

Driver	Backing	Notes
Mountpoint for S3 CSI	S3 bucket as a file system	Read-heavy large datasets (ML)

Operations

- **Expansion:** with `allowVolumeExpansion: true`, edit the PVC's `storage` request upwards; the CSI driver grows the disk and file system (shrinking is not supported).
- **Snapshots:** `VolumeSnapshotClass` + `VolumeSnapshot` (CSI snapshot controller); restore by creating a PVC with `dataSource` pointing at the snapshot.
- **Backups:** Velero (12.15) or AWS Backup for EBS/EFS.
- **Stuck PVC `Pending`:** no default StorageClass, wrong class name, CSI driver not installed or lacking IAM permissions, `WaitForFirstConsumer` (normal until a pod uses it), quota exceeded.
- **Pod stuck `ContainerCreating` with "Multi-Attach error":** an RWO volume is still attached to another node (old pod not yet gone); usually resolves after the old node detaches it.

12.10 Scheduling and availability

The **scheduler** places each pending pod in two phases: **filtering** (which nodes *can* run it: enough allocatable CPU/memory for the *requests*, node selectors, affinity, taints, volume zone, ports) then **scoring** (which feasible node is *best*: spreading, balanced usage, image locality). No feasible node = pod stays `Pending` with a `FailedScheduling` event explaining why.

Controls

Tool	Use
`nodeSelector`	Simple: only nodes with these labels (`kubernetes.io/arch: arm64`)
Node affinity	Expressive rules; `required...` (hard) or `preferred...` (soft)
Pod affinity / anti-affinity	Place near / away from other pods (e.g. no two replicas on one node)
Taints and tolerations	Nodes repel pods unless the pod tolerates the taint (dedicated GPU nodes, spot nodes, control plane)
Topology spread constraints	Spread replicas evenly across zones and nodes
PriorityClass	Important pods schedule first and can **preempt** (evict) lower-priority pods

```
# Fragment of a Deployment pod template: spread across AZs and nodes, prefer on-demand nodes
spec:
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule   # hard rule for zones
      labelSelector:
        matchLabels:
          app.kubernetes.io/name: api
    - maxSkew: 1
      topologyKey: kubernetes.io/hostname
      whenUnsatisfiable: ScheduleAnyway  # soft rule for nodes
      labelSelector:
        matchLabels:
          app.kubernetes.io/name: api
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 50
          preference:
            matchExpressions:
              - key: karpenter.sh/capacity-type
                operator: In
                values: ["on-demand"]
  tolerations:
    - key: dedicated
      operator: Equal
      value: shop
      effect: NoSchedule
```

Taint effects: `NoSchedule` (new pods without the toleration are not placed), `PreferNoSchedule` (avoid if possible), `NoExecute` (also evicts running pods without the toleration). Example: `kubectl taint nodes gpu-1 dedicated=ml:NoSchedule`; remove with a trailing `-`. Note: a toleration only **allows** a pod on a tainted node; to **force** it there, combine it with node affinity.

PodDisruptionBudget (PDB)

Limits **voluntary** disruptions (drain, upgrades, autoscaler scale-down), not crashes.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api
  namespace: shop
spec:
  minAvailable: 2                       # or maxUnavailable: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: api
```

A PDB that allows zero disruptions (e.g. `minAvailable: 1` with 1 replica) **blocks node drains and cluster upgrades** forever. Run at least 2 replicas when you add a PDB.

Node maintenance

```
kubectl cordon worker-2                                  # no new pods
kubectl drain worker-2 --ignore-daemonsets --delete-emptydir-data --timeout=10m
#   evicts pods respecting PDBs and graceful termination
# ... patch / replace the node ...
kubectl uncordon worker-2
```

High availability checklist for one service

- ≥ 2 replicas (3 for critical services), spread across **zones** and nodes.
- Readiness probes, graceful shutdown (SIGTERM + `preStop`), `maxUnavailable: 0` rollouts.
- A PDB.
- Requests set so the scheduler and autoscalers can do their job.
- No single-AZ dependency (EBS volume, single NAT gateway) hiding underneath.

12.11 Autoscaling

```
            traffic grows
                 │
HPA ──► more pod replicas ──► pods Pending (no room) ──► Cluster Autoscaler / Karpenter
                                                              ──► new nodes
VPA ──► right-sizes each pod's requests
KEDA ──► scales on events (queue length, Kafka lag, cron), including to zero
```

Horizontal Pod Autoscaler (HPA)

Needs **metrics-server** for CPU/memory (custom metrics need an adapter such as Prometheus Adapter or KEDA). Formula: `desired = ceil(current × currentMetric / target)`.

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api
  namespace: shop
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70         # % of the CPU *request*
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300    # avoid flapping
      policies:
        - type: Percent
          value: 50
          periodSeconds: 60
```

Rules: CPU utilization is relative to **requests** (no requests = HPA cannot work). Remove `spec.replicas` from the Deployment in Git once an HPA owns it, otherwise every deploy resets the count (or have GitOps ignore that field).

VPA, Cluster Autoscaler, Karpenter, KEDA

Tool	Scales	Notes
VPA	Pod requests/limits	Modes `Off` (recommendations only: safest and very useful), `Initial`, `Recreate` / `InPlaceOrRecreate`. Don't let VPA and HPA both act on CPU/memory of the same workload

Tool	Scales	Notes
Cluster Autoscaler	Node groups (ASGs)	Adds nodes when pods are Pending; removes underused nodes; works per node group with fixed instance types
Karpenter	Nodes directly (EC2 Fleet)	Picks instance types per pending pods, mixes spot/on-demand, consolidates to cheaper nodes, fast; configured with `NodePool` and `EC2NodeClass`. The common choice on EKS (EKS Auto Mode runs it for you)
KEDA	Deployments/Jobs on event sources	SQS queue depth, Kafka lag, Prometheus queries, cron; scale to zero

```
# Karpenter NodePool (v1 API): spot + on-demand, amd64/arm64, consolidation
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: general
spec:
  template:
    spec:
      nodeClassRef:
        group: karpenter.k8s.aws
        kind: EC2NodeClass
        name: default
      requirements:
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["spot", "on-demand"]
        - key: kubernetes.io/arch
          operator: In
          values: ["amd64", "arm64"]
      expireAfter: 720h                 # replace nodes every 30 days (fresh AMIs)
  limits:
    cpu: "200"                          # cost guard: never more than 200 vCPUs
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
    consolidateAfter: 1m
```

12.12 Security

Authentication and authorization

```
request ──► Authentication (who?) ──► Authorization (RBAC) ──► Admission (mutate+validate) ──► etcd
            certs, OIDC, cloud IAM      Role bindings              PSA, Kyverno, webhooks
```

Kubernetes has no user database: **humans** authenticate via certificates, OIDC (SSO) or cloud IAM (EKS access entries); **workloads** use **ServiceAccounts** (short-lived projected tokens).

RBAC

Object	Scope	Contains
Role	One namespace	Rules: apiGroups + resources + verbs
ClusterRole	Cluster-wide (or reusable in namespaces)	Same, plus cluster-scoped resources (nodes, PVs)
RoleBinding	One namespace	Grants a Role or ClusterRole to subjects in that namespace

Object	Scope	Contains
ClusterRoleBinding	Whole cluster	Grants a ClusterRole everywhere: use sparingly

Verbs: `get`, `list`, `watch`, `create`, `update`, `patch`, `delete`, `deletecollection`, plus special ones (`impersonate`, `escalate`, `bind`). Built-in ClusterRoles: `view`, `edit`, `admin`, `cluster-admin`.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: deployer
  namespace: shop
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deployer
  namespace: shop
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch", "patch", "update"]
  - apiGroups: [""]                       # "" = core API group
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: deployer
  namespace: shop
subjects:
  - kind: ServiceAccount
    name: deployer
    namespace: shop
  - kind: Group                           # e.g. an SSO group mapped via EKS access entries
    name: shop-developers
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: deployer
  apiGroup: rbac.authorization.k8s.io
```

```
kubectl auth can-i delete pods -n shop --as system:serviceaccount:shop:deployer
kubectl auth can-i --list -n shop --as alice
```

Dangerous permissions (treat as cluster-admin-equivalent): `secrets` get/list, `pods/exec`, creating pods in privileged namespaces, `escalate` / `bind` / `impersonate`, writing RBAC objects, `nodes/proxy`, wildcard `*` rules.

Pod Security Admission (PSA)

Built-in admission controller enforcing the **Pod Security Standards** per namespace via labels:

Level	Allows
`privileged`	Everything (system namespaces only)
`baseline`	Blocks known escalations: privileged, hostNetwork/hostPID, hostPath, added dangerous capabilities

Level	Allows
`restricted`	Baseline + must run as non-root, drop ALL capabilities, no privilege escalation, seccomp RuntimeDefault

```
apiVersion: v1
kind: Namespace
metadata:
  name: shop
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: latest
    pod-security.kubernetes.io/warn: restricted
    pod-security.kubernetes.io/audit: restricted
```

Policy engines and admission control

Tool	Language	Typical rules
ValidatingAdmissionPolicy (built-in, GA 1.30)	CEL expressions	Simple validations without webhooks
Kyverno	YAML policies	Require labels/requests/probes, disallow `:latest`, only allowed registries, verify image signatures, **mutate** (add defaults) and **generate** (default NetworkPolicy per namespace)
OPA **Gatekeeper**	Rego	Same goals, powerful for complex logic

```
# Kyverno: images must come from our ECR and must not use ":latest"
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: trusted-images
spec:
  validationFailureAction: Enforce
  rules:
    - name: only-our-registry
      match:
        any:
          - resources:
              kinds: ["Pod"]
      validate:
        message: "Images must come from 123456789012.dkr.ecr.ap-south-1.amazonaws.com"
        pattern:
          spec:
            containers:
              - image: "123456789012.dkr.ecr.ap-south-1.amazonaws.com/*"
    - name: no-latest-tag
      match:
        any:
          - resources:
              kinds: ["Pod"]
      validate:
        message: "Use an immutable tag, not :latest"
        pattern:
          spec:
            containers:
              - image: "!*:latest"
```

Roll out policies in **audit** mode first, fix violations, then enforce.

Cluster security checklist (senior view)

Area	Controls
API access	Private or IP-restricted endpoint, SSO/IAM identities, no shared kubeconfigs, short-lived credentials
RBAC	Least privilege per team/namespace, no `cluster-admin` for humans day to day, audit bindings regularly
Workloads	PSA `restricted`, non-root, read-only root FS, no privileged/host access, resource limits
Network	Default-deny NetworkPolicies, private nodes, security groups, egress control
Supply chain	Trusted registries, image scanning, signed images (cosign) verified by admission (Kyverno/Sigstore policy-controller), SBOMs
Secrets	External secret store, encryption at rest, no secrets in Git or env dumps
Workload identity	IRSA / EKS Pod Identity per ServiceAccount; block pod access to the node's instance metadata (IMDSv2 hop limit 1)
Detection	API **audit logs** (EKS control plane logging), runtime detection (**Falco**, GuardDuty EKS runtime monitoring)
Benchmarks	**kube-bench** (CIS Kubernetes Benchmark), Kubescape/Trivy cluster scans
Lifecycle	Upgrade Kubernetes and nodes regularly (patches), minimal node OS (Bottlerocket)

12.13 Packaging: Helm and Kustomize

Raw YAML repeats itself across environments. Two standard tools solve that:

	Helm	Kustomize
Approach	Templates (Go templates) + values files → YAML	Plain YAML **base** + **overlays** with patches (no templating)
Packaging	Charts, versioned, published to repositories (OCI registries)	Directories in Git
Release tracking	Yes: install/upgrade/rollback history (stored as Secrets)	No: just `kubectl apply -k`
Best for	Distributing software (third-party apps), complex parameterization	Your own apps, environment differences
Built in	Separate CLI	`kubectl apply -k` / `kustomize build`

Many teams use **both**: Helm for third-party components (ingress controllers, Prometheus, cert-manager) and Kustomize or a small internal chart for their own services, all deployed by Argo CD.

Helm

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm search repo kube-prometheus-stack --versions | head
helm show values prometheus-community/kube-prometheus-stack > default-values.yaml
helm upgrade --install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace --version 65.1.0 -f values-prod.yaml \
  --atomic --timeout 10m                # roll back automatically if the upgrade fails
helm list -A
helm history monitoring -n monitoring
helm rollback monitoring 3 -n monitoring
helm uninstall monitoring -n monitoring
helm template ./charts/api -f values-dev.yaml   # render locally (CI checks, GitOps)
helm lint ./charts/api
helm install api oci://123456789012.dkr.ecr.ap-south-1.amazonaws.com/charts/api --version 1.2.0
```

Always **pin chart versions** (`--version`) and keep values files in Git.

Chart structure:

```
charts/api/
├── Chart.yaml            # name, version (chart), appVersion (app), dependencies
├── values.yaml           # defaults
├── values.schema.json    # optional: validates values
├── templates/
│   ├── _helpers.tpl      # named templates (labels, names)
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── hpa.yaml
│   └── NOTES.txt         # printed after install
└── charts/               # dependency charts
```

```
# Chart.yaml
apiVersion: v2
name: api
description: Shop API
type: application
version: 1.2.0                        # chart version: bump on every chart change
appVersion: "1.4.2"                   # default image tag
```

```
# values.yaml
replicaCount: 2
image:
  repository: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api
  tag: ""                             # empty = use appVersion
resources:
  requests:
    cpu: 250m
    memory: 256Mi
  limits:
    memory: 256Mi
autoscaling:
  enabled: false
  maxReplicas: 10
```

```
# templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "api.fullname" . }}
  labels:
    {{- include "api.labels" . | nindent 4 }}
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
  {{- end }}
  selector:
    matchLabels:
      {{- include "api.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "api.selectorLabels" . | nindent 8 }}
      annotations:                      # restart pods when the ConfigMap changes
        checksum/config: >-
          {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
    spec:
      containers:
        - name: app
          {{- $tag := .Values.image.tag | default .Chart.AppVersion }}
          image: "{{ .Values.image.repository }}:{{ $tag }}"
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
```

Template essentials: `.Values`, `.Release.Name`, `.Release.Namespace`, `.Chart.Version`; functions `default`, `quote`, `toYaml`, `nindent`, `required "msg"`, `include`, `tpl`; control `if/else`, `range`, `with`. **Hooks** (`helm.sh/hook: pre-upgrade`) run Jobs such as migrations. Inspect what will change with the `helm diff` plugin before upgrading.

Kustomize

```
k8s/
├── base/
│   ├── kustomization.yaml
│   ├── deployment.yaml
│   └── service.yaml
└── overlays/
    ├── dev/kustomization.yaml
    └── prod/
        ├── kustomization.yaml
        └── replicas-patch.yaml
```

```
# base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
labels:
  - pairs:
      app.kubernetes.io/part-of: shop
    includeSelectors: false
```

```
# overlays/prod/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: shop-prod
resources:
  - ../../base
images:
  - name: shop/api                        # image name used in base/deployment.yaml
    newName: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api
    newTag: "1.4.2"                       # CI/GitOps bumps this line
patches:
  - path: replicas-patch.yaml
configMapGenerator:
  - name: api-config
    literals:
      - LOG_LEVEL=warn                    # generated name gets a hash: change = rolling restart
```

```
# overlays/prod/replicas-patch.yaml (strategic merge patch)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 4
```

```
kubectl kustomize overlays/prod          # render
kubectl diff -k overlays/prod            # compare with the cluster
kubectl apply -k overlays/prod
kustomize edit set image shop/api=REGISTRY/shop/api:1.4.3   # what CI runs to bump a tag
```

Validating manifests in CI

Tool	Checks
`kubeconform`	Schema validity against the Kubernetes version (and CRD schemas)
`kube-linter` / Polaris / Datree-style rules	Best practices: probes, requests, non-root, no `latest`
`helm lint`, `helm template | kubeconform`	Chart correctness
Kyverno CLI / `conftest` (OPA)	Your organization's policies before they reach the cluster
`pluto` / `kubent`	Deprecated/removed API versions before upgrades

12.14 Observability on Kubernetes

Signal	Standard tools
Resource metrics	**metrics-server** (`kubectl top` , HPA)
Metrics	**Prometheus** (kube-prometheus-stack: Prometheus Operator, Alertmanager, Grafana, node-exporter, **kube-state-metrics**) or Amazon Managed Prometheus; `ServiceMonitor` / `PodMonitor` CRs select what to scrape
Logs	Containers log to **stdout/stderr**; kubelet stores them under `/var/log/pods` ; a DaemonSet (Fluent Bit, Vector, Grafana Alloy) ships them to Loki, OpenSearch or CloudWatch

Signal	Standard tools
Traces	OpenTelemetry SDKs + OpenTelemetry Collector → Tempo, Jaeger, X-Ray
Events	`kubectl get events` (kept about one hour by default): export them (event exporter) for history
AWS	**CloudWatch Container Insights** (EKS add-on), control plane logs (api, audit, authenticator)

Key metrics to know: pod restarts (`kube_pod_container_status_restarts_total`), pods not ready, Deployment replicas available vs desired, container CPU throttling (`container_cpu_cfs_throttled_periods_total`), memory working set vs limit, OOM kills, node pressure conditions, PVC usage, API server latency/errors. Full coverage in the Monitoring & SRE notes.

12.15 Cluster operations

Namespaces as tenancy units: quotas and defaults

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: shop-quota
  namespace: shop
spec:
  hard:
    requests.cpu: "20"
    requests.memory: 40Gi
    limits.memory: 60Gi
    pods: "100"
    persistentvolumeclaims: "20"
    services.loadbalancers: "2"
---
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  namespace: shop
spec:
  limits:
    - type: Container
      defaultRequest:                    # applied when a container sets no requests
        cpu: 100m
        memory: 128Mi
      default:                           # applied when a container sets no limits
        memory: 256Mi
      max:
        memory: 4Gi
```

With a quota on `requests.cpu`, pods **without** requests are rejected; a LimitRange supplies defaults so they are accepted.

Upgrades

Kubernetes releases a minor version roughly every **4 months**; each is supported about 14 months upstream (EKS: 14 months standard support, then paid extended support for 12 more).

1. Read the release notes and the **deprecated/removed APIs** list; scan manifests and Helm releases (`pluto`, `kubent`, EKS upgrade insights).
2. Upgrade in **dev/staging first**.
3. Control plane first, **one minor version at a time** (1.32 → 1.33 → 1.34).
4. Then add-ons (CoreDNS, kube-proxy, VPC CNI, CSI drivers, ingress controller) to compatible versions.

5. Then nodes: new node group/AMI or Karpenter drift replaces nodes gradually; PDBs protect availability. Nodes (kubelet) may lag the control plane by up to three minor versions, never be newer.
6. Verify workloads, dashboards and alerts.

Alternative for big jumps or high risk: **blue/green clusters** (build a new cluster, move traffic).

Backup and disaster recovery

What	How
Cluster objects	**GitOps** is the primary backup: everything can be re-applied from Git. **Velero** backs up API objects (and PV snapshots) to S3 for the rest
etcd (self-managed)	`etcdctl snapshot save` on a schedule, stored off-cluster; test restores. Managed control planes (EKS) handle etcd
Persistent data	CSI snapshots, Velero, AWS Backup, database-native backups (prefer managed DBs)
Rebuild	Terraform for the cluster, GitOps for workloads: measure how long a full rebuild takes (RTO)

```
velero backup create shop-daily --include-namespaces shop --ttl 720h
velero schedule create shop-nightly --schedule "0 2 * * *" --include-namespaces shop
velero restore create --from-backup shop-daily
```

Extending Kubernetes: CRDs and operators

A **CustomResourceDefinition (CRD)** adds a new object type to the API (`Certificate`, `ExternalSecret`, `Application`, `NodePool`). An **operator** is a controller that reconciles those objects with domain knowledge: "a `Postgres` object with 3 instances" becomes StatefulSets, Services, backups and failover. You build them with Kubebuilder / Operator SDK (Go) or Kopf (Python). Much of the platform you run (cert-manager, ESO, Argo CD, Karpenter, Prometheus Operator) is operators.

```
kubectl get crds
kubectl explain certificate.spec          # CRDs with schemas are documented too
```

12.16 Amazon EKS in practice

Area	What to know
Control plane	Managed by AWS across 3 AZs; you pay a per-cluster hourly fee (about $0.10/h standard support, more for extended support) plus nodes
Compute options	**Managed node groups** (EC2 ASGs AWS manages), **Karpenter** (self-managed), **Fargate** (serverless pods, no DaemonSets), **EKS Auto Mode** (AWS runs Karpenter, VPC CNI, EBS CSI, load balancer controller for you)
Networking	**VPC CNI**: pods use VPC IPs, so size subnets generously (e.g. /19 per AZ for pods) or enable prefix delegation; nodes in **private subnets**; public/private API endpoint options
Human access	**EKS access entries** map IAM principals to Kubernetes permissions (access policies or groups for RBAC); the old `aws-auth` ConfigMap is deprecated
Pod → AWS access	**EKS Pod Identity** (agent add-on, association: namespace + service account → IAM role) or **IRSA** (OIDC provider + annotated ServiceAccount). Never use node instance-role permissions for apps
Load balancing	**AWS Load Balancer Controller**: Ingress → ALB, Service `LoadBalancer` → NLB, `ip` target type for direct pod targets

Area	What to know
Storage	EBS CSI and EFS CSI drivers as EKS add-ons (with IAM via Pod Identity)
Add-ons	Managed versions of VPC CNI, CoreDNS, kube-proxy, EBS CSI, Pod Identity Agent, metrics-server, CloudWatch observability
Logging	Enable control plane logs (api, audit, authenticator) to CloudWatch
Cost	Right-size requests, Karpenter consolidation, spot for stateless workloads, Graviton (arm64) nodes, avoid cross-AZ chatter, delete idle LBs/volumes

```
aws eks update-kubeconfig --region ap-south-1 --name shop-prod   # writes kubeconfig context
aws eks list-access-entries --cluster-name shop-prod
aws eks create-pod-identity-association --cluster-name shop-prod \
  --namespace shop --service-account api --role-arn arn:aws:iam::123456789012:role/shop-api
```

Creating a cluster is normally **Terraform** (`terraform-aws-modules/eks/aws`) or `eksctl` :

```
# eksctl cluster config (lab use; production via Terraform)
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata:
  name: lab
  region: ap-south-1
  version: "1.33"
managedNodeGroups:
  - name: default
    instanceType: t3.medium
    desiredCapacity: 2
    minSize: 1
    maxSize: 3
    privateNetworking: true
addons:
  - name: vpc-cni
  - name: coredns
  - name: kube-proxy
  - name: eks-pod-identity-agent
```

Cost warning: an EKS cluster costs about $0.10 per hour (≈ $73/month) **before** nodes, NAT gateways and load balancers. It is not covered by "always free" offers. Labs: create, practice, and **delete the same day** (`eksctl delete cluster --name lab --region ap-south-1` , then check for leftover load balancers, EBS volumes and NAT gateways). Do most practice on **kind** for free.

12.17 Complete production application manifests

One service, `api` in namespace `shop` , with everything a production review expects. Apply with `kubectl apply -f shop-api.yaml` (in real teams: through Kustomize/Helm and Argo CD).

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: shop
  labels:
    pod-security.kubernetes.io/enforce: restricted
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: api
  namespace: shop
automountServiceAccountToken: false
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: api-config
  namespace: shop
data:
  LOG_LEVEL: info
  DB_HOST: shop-db.abc123.ap-south-1.rds.amazonaws.com
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: shop
  labels:
    app.kubernetes.io/name: api
    app.kubernetes.io/part-of: shop
spec:
  revisionHistoryLimit: 5
  selector:
    matchLabels:
      app.kubernetes.io/name: api
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app.kubernetes.io/name: api
        app.kubernetes.io/part-of: shop
    spec:
      serviceAccountName: api
      terminationGracePeriodSeconds: 30
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        seccompProfile:
          type: RuntimeDefault
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app.kubernetes.io/name: api
      containers:
        - name: app
          image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2
          ports:
            - name: http
              containerPort: 8080
```

```
          envFrom:
            - configMapRef:
                name: api-config
          env:
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-credentials   # created by External Secrets, never stored in Git
                  key: DB_PASSWORD
          resources:
            requests:
              cpu: 250m
              memory: 256Mi
            limits:
              memory: 256Mi
          startupProbe:
            httpGet:
              path: /healthz
              port: http
            failureThreshold: 30
            periodSeconds: 2
          readinessProbe:
            httpGet:
              path: /ready
              port: http
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /healthz
              port: http
            periodSeconds: 10
          lifecycle:
            preStop:
              exec:
                command: ["sleep", "5"]
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
          volumeMounts:
            - name: tmp
              mountPath: /tmp
      volumes:
        - name: tmp
          emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: shop
spec:
  selector:
    app.kubernetes.io/name: api
  ports:
    - name: http
      port: 80
      targetPort: http
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api
  namespace: shop
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
```

```
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/healthcheck-path: /ready
spec:
  ingressClassName: alb
  rules:
    - host: api.shop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  name: http
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api
  namespace: shop
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 3
  maxReplicas: 15
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api
  namespace: shop
spec:
  maxUnavailable: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: api
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api
  namespace: shop
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: api
  policyTypes: ["Ingress"]
  ingress:
    - ports:                              # ALB (ip targets) reaches pods from VPC addresses
        - protocol: TCP
          port: 8080
      from:
        - ipBlock:
            cidr: 10.0.0.0/16
```

Review checklist this file satisfies: no `replicas` field (HPA owns it) · immutable image tag · requests and memory limit · three probes · graceful shutdown · non-root, read-only, no capabilities · dedicated ServiceAccount without a token · secret from an external store · zone spread · PDB · NetworkPolicy · Pod Security `restricted` namespace.

12.18 Troubleshooting guide

The method

```
1. What is broken, for whom, since when?    kubectl get pods -n shop -o wide; recent deploys?
2. Look at the object and its EVENTS         kubectl describe <pod|deploy|svc|node|pvc>
3. Look at logs                              kubectl logs <pod> [-c c] [--previous]
4. Follow the chain                          Ingress → Service → EndpointSlice → Pod → container
5. Check the node/cluster level              kubectl get nodes; describe node; kube-system pods
6. Change one thing, verify, write it down
```

Pod status reference

Status / reason	Meaning	Check	Typical fix
`Pending` + `FailedScheduling` "Insufficient cpu/memory"	No node has room for the **requests**	`describe pod`, `kubectl describe nodes` (Allocated resources)	Lower requests, add nodes / check autoscaler
`Pending` "didn't match node affinity/selector", "untolerated taint"	Placement rules exclude all nodes	Pod spec vs node labels/taints	Fix labels, selectors, tolerations
`Pending` "unbound PersistentVolumeClaim"	Storage not provisioned	`describe pvc`, CSI driver pods/logs	StorageClass name, CSI driver/IAM, AZ
`ContainerCreating` (long)	Volume mount, CNI or secret problem	Events: `FailedMount`, `FailedCreatePodSandBox`	Missing ConfigMap/Secret, CNI IPs exhausted (VPC CNI), Multi-Attach
`ErrImagePull` / `ImagePullBackOff`	Image can't be pulled	Events: "not found", "unauthorized", "timeout"	Tag/name typo, registry auth (`imagePullSecrets`, node IAM for ECR), network/NAT to registry
`CrashLoopBackOff`	Container starts and exits repeatedly	`logs --previous`, exit code in `describe`	App error, missing config/env, wrong command, failing dependency, liveness too aggressive
`OOMKilled` (exit 137)	Memory above **limit**	`describe` Last State, memory metrics	Fix leak or raise limit; tune runtime heap (JVM `-XX:MaxRAMPercentage`)
Exit code 1 / 2	Application error / misuse of shell	Logs	Fix the app/config
Exit code 126 / 127	Command not executable / not found	Image contents, `command`	Correct path, add binary, `chmod +x`

Status / reason	Meaning	Check	Typical fix
Exit code 143	SIGTERM (normal at shutdown)	—	If during start: probes killing it
`CreateContainerConfigError`	Referenced ConfigMap/Secret/key missing	Events	Create it or fix the key name
`Running` but `0/1 READY`	Readiness probe failing	`describe` (probe failure messages), app logs	Probe path/port wrong, app not ready, dependency down
`Terminating` forever	Finalizer or unreachable node	`get pod -o yaml` (finalizers), node status	Fix the controller / node; last resort `--grace-period=0 --force`
`Evicted`	Node pressure (memory/disk)	`describe node` conditions	Set requests/limits, clean disk, bigger nodes

Service and networking

```
kubectl get svc api -n shop -o wide                          # selector, ports
kubectl get endpointslices -n shop -l kubernetes.io/service-name=api   # empty = no ready pods
kubectl get pods -n shop -l app.kubernetes.io/name=api --show-labels  # do labels match?
kubectl run -it --rm net --image=nicolaka/netshoot --restart=Never -n shop -- bash
#   inside: curl -sv http://api/ready ; nslookup api ; nc -zv api 80
kubectl port-forward -n shop deploy/api 8080:8080            # bypass Service: test the pod
```

Symptom	Likely cause
Service has no endpoints	Selector/label mismatch, pods not Ready, wrong namespace
Connection refused via Service	`targetPort` ≠ container port, app listening on `127.0.0.1` instead of `0.0.0.0`
Timeout between pods	NetworkPolicy denying, security groups (VPC CNI / security groups for pods), node firewall
DNS fails (`could not resolve`)	CoreDNS pods down, NetworkPolicy blocking port 53, wrong name/namespace
Ingress 404	Host/path rule not matching, wrong `ingressClassName`, controller not running
Ingress/ALB 502/503/504	No healthy targets (readiness/health check path), target port wrong, app timeouts, security groups
LoadBalancer `EXTERNAL-IP` stays `<pending>`	No load balancer controller, missing subnet tags (`kubernetes.io/role/elb`), IAM permissions

Deployments and rollouts

Symptom	Check
`rollout status` hangs	New pods not Ready: `describe` new pods; `progressDeadlineSeconds` exceeded → `ProgressDeadlineExceeded`
Change "did nothing"	Edited only a ConfigMap (pods don't restart) or the image tag is the same mutable tag

Symptom	Check
Old version back after a deploy	GitOps reverted a manual `kubectl` edit (Git is the source of truth)
Replicas jump back	HPA owns `replicas` and Git also sets it

Nodes

```
kubectl get nodes -o wide
kubectl describe node worker-2   # Conditions: Ready, Memory/Disk/PIDPressure
kubectl get pods -A --field-selector spec.nodeName=worker-2
kubectl debug node/worker-2 -it --image=busybox:1.36    # then: chroot /host; journalctl -u kubelet
```

`NotReady` causes: kubelet stopped or can't reach the API, container runtime down, CNI failing, disk full, expired certificates, instance problems (check EC2 status checks).

Access (RBAC) errors

`Error from server (Forbidden): pods is forbidden: User "alice" cannot list resource "pods" in the namespace "shop"` → the identity is authenticated but not authorized. Check: `kubectl auth can-i list pods -n shop --as alice`, RoleBindings in the namespace, EKS access entries. `error: You must be logged in to the server (Unauthorized)` → authentication: expired token, wrong AWS profile, missing access entry.

12.19 Hands-on labs (kind on your laptop: free)

#	Lab	You prove that you can
1	Create a 3-node kind cluster (12.3); explore with `get nodes`, `get pods -A`, `api-resources`	Run and inspect a cluster
2	Deploy nginx with 3 replicas; delete a pod and watch it come back (`-w`)	Explain reconciliation
3	Roll out a new image; break it with a bad tag; read events; `rollout undo`	Debug and roll back
4	Expose with a ClusterIP Service; test from a netshoot pod; break the selector and find it via EndpointSlices	Debug Services
5	Add probes; make readiness fail; observe `0/1 READY` and empty endpoints	Use probes correctly
6	Set a 64Mi memory limit and run a memory hog (`polinux/stress`); see `OOMKilled`	Understand limits and QoS
7	ConfigMap + Secret as env and files; change them and observe what updates	Manage configuration
8	StatefulSet with a PVC (kind's default `standard` class); delete pod 0 and confirm data survives	Use persistent storage
9	Install ingress-nginx or Envoy Gateway; route two paths to two Services	Route HTTP traffic
10	Install metrics-server (with `--kubelet-insecure-tls` on kind only) + HPA; generate load; watch scaling	Autoscale
11	Create a namespace with PSA `restricted`; try to run a root pod; fix the spec	Apply Pod Security
12	Role + RoleBinding for a ServiceAccount; test with `auth can-i --as`	Configure RBAC
13	Package lab 2–5 as a Helm chart and as Kustomize base/overlays (dev/prod)	Package apps
14	Default-deny + allow rules with Calico or Cilium installed in kind	Use NetworkPolicy

#	Lab	You prove that you can
15	Troubleshooting drill: a partner (or script) breaks image, selector, probe, ConfigMap key, quota; fix each from events	Troubleshoot methodically

Clean up: `kind delete cluster --name lab`.

12.20 Interview questions

1. What problem does Kubernetes solve compared with running Docker on servers?
2. Explain the control plane components and what each does.
3. Walk through what happens after `kubectl apply -f deployment.yaml` until traffic reaches the pod.
4. What is reconciliation / a controller? Give an example.
5. Pod vs container vs Deployment vs ReplicaSet.
6. Why shouldn't you create bare pods in production?
7. Liveness vs readiness vs startup probes; what goes wrong when liveness checks the database?
8. Requests vs limits; what happens when a container exceeds each? QoS classes.
9. Why do many teams avoid CPU limits?
10. How does a rolling update work? Explain `maxSurge` and `maxUnavailable`. How do you roll back?
11. How do you achieve zero-downtime deployments (probes, preStop, SIGTERM, PDB)?
12. Deployment vs StatefulSet vs DaemonSet vs Job/CronJob: when each?
13. Service types; what is a headless Service?
14. How does a Service find its pods? What does empty EndpointSlices tell you?
15. Explain Kubernetes DNS names and `ndots`.
16. Ingress vs LoadBalancer Service vs Gateway API.
17. How do NetworkPolicies work? Why add a DNS egress rule after default-deny?
18. ConfigMap vs Secret; are Secrets secure? How do you manage secrets in production?
19. PV, PVC and StorageClass; RWO vs RWX; what is `WaitForFirstConsumer` for on AWS?
20. Taints/tolerations vs node affinity; topology spread constraints.
21. What is a PodDisruptionBudget, and how can it block an upgrade?
22. HPA vs VPA vs Cluster Autoscaler vs Karpenter vs KEDA.
23. Explain RBAC objects and how to grant a CI pipeline deploy rights to one namespace.
24. What are Pod Security Standards; what does `restricted` require?
25. How would you enforce "only images from our registry, no `latest`"?
26. Helm vs Kustomize; how do you make pods restart when config changes?
27. How do you upgrade an EKS cluster safely?
28. How do pods get AWS permissions on EKS (Pod Identity / IRSA) and why not the node role?
29. VPC CNI: why can pods run out of IP addresses, and how do you fix it?
30. What are CRDs and operators? Name ones you've used.
31. How do you back up and restore a cluster?
32. A pod is `CrashLoopBackOff`: your exact steps.
33. A pod is `Pending`: list the possible causes.
34. Users get 502 from the ALB after a deploy: how do you investigate?
35. How do you make a multi-tenant cluster safe for several teams (namespaces, RBAC, quotas, policies, NetworkPolicies)?

12.21 Self-check

- [ ] I can explain every control plane and node component and the apply → running flow.

- [ ] I can write a Deployment, Service, Ingress, ConfigMap, Secret reference, HPA, PDB and NetworkPolicy from memory.
- [ ] I can choose between Deployment, StatefulSet, DaemonSet, Job and CronJob and justify it.
- [ ] I can configure probes, resources and graceful shutdown correctly and explain QoS.
- [ ] I can trace traffic Ingress → Service → EndpointSlice → Pod and debug each hop.
- [ ] I can set up storage with StorageClass/PVC and explain AZ limits of EBS.
- [ ] I can control placement (affinity, taints, spread) and availability (PDB, drain).
- [ ] I can configure HPA and explain Karpenter/Cluster Autoscaler.
- [ ] I can create least-privilege RBAC and apply Pod Security `restricted`.
- [ ] I can package an app with Helm and with Kustomize overlays.
- [ ] I can explain an EKS setup: access entries, Pod Identity, LB controller, VPC CNI, add-ons, upgrades, costs.
- [ ] I can diagnose every pod status in 12.18 from events and logs.

REMEMBER

- Debug order: **get → describe (Events) → logs --previous → exec → get endpoints**.
- **Labels connect everything**: Deployment selector ↔ pod labels ↔ Service selector.
- **Readiness** = send traffic? **Liveness** = restart? **Requests** = scheduling, **limits** = hard cap.